

CustomizeHandler 최적화

- 이슈파와
 - 현재 플레이어를 스폰할때 SkeletalMeshLoad 함수가 3번 불림
 - 내부에 같은 Index(Key) 일때 리턴하지만 3가지의 함수호출마다 다른 키값을 세팅하기에 무용지물인 상태
 - Set / SetBodyChange / SetCostume
 - Set / SetBodyChange / SetCostume

```
const FGsSchemaUserData* userData = UGTableUtil::FindRow<FGsSchemaUserData>(_characterIndex)._creatureId);
_customizeHandler->ChangeWeaponSocket(userData, _listCharacter[Index]._classType, true);

_customizeHandler->Set(_listCharacter[_characterIndex]._faceData, [_characterIndex]._bodyData);

uint32 weaponPartsID = GetPartsMeshID(EGsPartsType::WEAPON, _listCharacter[_characterIndex]);
uint32 armorID = GetPartsMeshID(EGsPartsType::BODY, _listCharacter[_characterIndex]);
int32 weaponId = _listCharacter[_characterIndex]._weaponId;
CreatureWeaponType weaponType = CreatureWeaponType::NONE;
const FGsSchemaItemEquipDetail* weapon = UGItemManager::GetEquipItemTableDataByTID(weaponId);
if (weapon)
{
    weaponType = weapon->weaponType;
}

_customizeHandler->SetWeaponChange(weaponPartsID, weaponType, _listCharacter[_characterIndex]._weaponLevel);
_customizeHandler->SetBodyChange(armorID);
_customizeHandler->SetSkinColorChange(_listCharacter[_characterIndex]._faceData.mSkinColorIndex);
_customizeHandler->SetNakedShape(_listCharacter[_characterIndex]._faceData, _listCharacter[_characterIndex]._bodyData);
_customizeHandler->SetCostume(_listCharacter[_characterIndex]._lookInfo.mCostumeId, _listCharacter[_characterIndex]._lookInfo.mVie
```

커스터 마이즈 내부 함수

FGsCustomizeHandler::Set(const CustomizeFaceData& InFaceData, const CustomizeBodyData& InBodyData, bool InDirect)

```
void FGsCustomizeHandler::Set(const CustomizeFaceData& InFaceData, const CustomizeBodyData& InBodyData, bool InDirect)
{
    SetNakedShape(InFaceData, InBodyData);
    // face
    SetHairChange(InFaceData.mHairId, InDirect);
    SetFaceChange(InFaceData.mFaceId, InDirect);
    SetBodyChange(InBodyData.mBodyId, InDirect);
    SetTattooChange(InFaceData.mTattooId, InDirect);

    SetBodyBoneScaleChange(EGsUICustomizeType::HEIGHT, InBodyData.mHeight);
    SetBodyBoneScaleChange(EGsUICustomizeType::HEAD_SIZE, InBodyData.mHeadSize);
    SetBodyBoneScaleChange(EGsUICustomizeType::CHEST_SIZE, InBodyData.mChestSize);
    SetBodyBoneScaleChange(EGsUICustomizeType::WAIST_SIZE, InBodyData.mWaistSize);
    SetBodyBoneScaleChange(EGsUICustomizeType::PELVIS_SIZE, InBodyData.mPelvisSize);
    SetBodyBoneScaleChange(EGsUICustomizeType::LEG_SIZE, InBodyData.mLegSize);

    SetSkinColorChange(InBodyData.mSkinColorIndex);
    SetHairColorChange(InFaceData.mHairColorIndex);
    SetPupilColorChange(InFaceData.mPupilColorIndex);
    SetBrowColorChange(InFaceData.mBrowColorIndex);

    RecalcCapsuleSize();
}
```

SetCostume 함수

InvalidateCostume

```
auto InvalidateCostume = [&]()->void
{
    if (auto costume = FGsCostumeData::loadCostume(costumeId))
    {
        _costumeParts.hairCostumeId = costume->GetCostumeHairID(_genderType);
        _costumeParts.faceCostumeId = costume->GetCostumeFaceID(_genderType);
        _costumeParts.bodyCostumeId = costume->GetCostumeClothID(_genderType);

        auto hairId = _costumeParts.hairCostumeId == 0 || !hairIsOn ? _feature.faceShape.mHairId : _costumeParts.hairCostumeId;
        auto faceId = _costumeParts.faceCostumeId == 0 ? _feature.faceShape.mFaceId : _costumeParts.faceCostumeId;
        auto bodyId = _costumeParts.bodyCostumeId == 0 ? _feature.bodyShape.mBodyId : _costumeParts.bodyCostumeId;

        if (hairId > 0)
        {
            SetHairChange(hairId, true);
            SetHairColorChange(_feature.faceShape.mHairColorIndex);
        }
        if (faceId > 0)
        {
            SetFaceChange(faceId, true);
            SetBodyBoneScaleChange(EGsUICustomizeType::HEAD_SIZE, _feature.bodyShape.mHeadSize);
        }
        if (bodyId > 0)
        {
            auto pelvisSize = costume->IsCustomizedPelvisSizeApply() ? _feature.bodyShape.mPelvisSize : DEFAULT_CUSTOM_VALUE;

            SetBodyChange(bodyId, true);

            if (BodyScaleDefault)
            {
                SetBodyBoneScaleChange(EGsUICustomizeType::HEIGHT, DEFAULT_CUSTOM_VALUE);
                SetBodyBoneScaleChange(EGsUICustomizeType::CHEST_SIZE, DEFAULT_CUSTOM_VALUE);
                SetBodyBoneScaleChange(EGsUICustomizeType::WAIST_SIZE, DEFAULT_CUSTOM_VALUE);
                SetBodyBoneScaleChange(EGsUICustomizeType::PELVIS_SIZE, DEFAULT_CUSTOM_VALUE);
                SetBodyBoneScaleChange(EGsUICustomizeType::LEG_SIZE, DEFAULT_CUSTOM_VALUE);
            }
            else
            {
                SetBodyBoneScaleChange(EGsUICustomizeType::HEIGHT, _feature.bodyShape.mHeight);
                SetBodyBoneScaleChange(EGsUICustomizeType::CHEST_SIZE, _feature.bodyShape.mChestSize);
                SetBodyBoneScaleChange(EGsUICustomizeType::WAIST_SIZE, _feature.bodyShape.mWaistSize);
                SetBodyBoneScaleChange(EGsUICustomizeType::PELVIS_SIZE, pelvisSize);
                SetBodyBoneScaleChange(EGsUICustomizeType::LEG_SIZE, _feature.bodyShape.mLegSize);
            }
        }
    }
}
```

```

        if (faceId > 0 || bodyId > 0)
        {
            SetSkinColorChange(_feature.bodyShape.mSkinColorIndex);

            if (_costumeParts.faceCostumeId == 0) //얼굴 갈아끼우기 안되었을 때
            {
                SetPupilColorChange(_feature.faceShape.mPupilColorIndex);
                SetBrowColorChange(_feature.faceShape.mBrowColorIndex);
                SetTattooChange(_feature.faceShape.mTattooId, true);
            }

            SetCostumeEffect(costume->_costumeData->effectInfo.path,
                costume->_costumeData->effectInfo.socketName, costume->_costumeData->effectInfo.socketTransform);

            costume.Reset();
        }
    };
}

```

◦ 초기화 시 1번만 세팅할 것도 매번 처리중

FGsCustomizeHandler::InitializeActor(AActor* Owner)

void FGsCustomizeHandler::InitializeActor(AActor* Owner)

```

{
    _characterPlayer = Cast<AGsCharacterPlayer>(Owner);

    _targetNames.Reset();
    _targetNames.Add(EGsTargetNames::CheekSize, TargetNameCheekSize);
    _targetNames.Add(EGsTargetNames::ChinLength, TargetNameChinLength);
    _targetNames.Add(EGsTargetNames::ChinSize, TargetNameChinSize);
    _targetNames.Add(EGsTargetNames::NosePosition, TargetNameNosePosition);
    _targetNames.Add(EGsTargetNames::EyePosition, TargetNameEyePosition);
    _targetNames.Add(EGsTargetNames::EyeSpacing, TargetNameEyeSpacing);
    _targetNames.Add(EGsTargetNames::EyeSize, TargetNameEyeSize);
    _targetNames.Add(EGsTargetNames::BrowPosition, TargetNameBrowPosition);
    _targetNames.Add(EGsTargetNames::BrowAngle, TargetNameBrowAngle);
    _targetNames.Add(EGsTargetNames::BrowLength, TargetNameBrowLength);
    _targetNames.Add(EGsTargetNames::MouthPosition, TargetNameMouthPosition);
    _targetNames.Add(EGsTargetNames::MouthSize, TargetNameMouthSize);

    _targetNames.Add(EGsTargetNames::Height, TargetNameHeight);
    _targetNames.Add(EGsTargetNames::Head, TargetNameHead);
    _targetNames.Add(EGsTargetNames::Chest, TargetNameChest);
    _targetNames.Add(EGsTargetNames::WaistR, TargetNameWaistR);
    _targetNames.Add(EGsTargetNames::WaistL, TargetNameWaistL);
    _targetNames.Add(EGsTargetNames::Pelvis, TargetNamePelvis);
    _targetNames.Add(EGsTargetNames::LegsR, TargetNameLegsR);
    _targetNames.Add(EGsTargetNames::LegsL, TargetNameLegsL);

    _targetNames.Add(EGsTargetNames::HairColor, TargetNameHairColor);
    _targetNames.Add(EGsTargetNames::HairShadowColor, TargetNameHairShadowColor);
    _targetNames.Add(EGsTargetNames::HairSpecularColor, TargetNameHairSpecularColor);

    _targetNames.Add(EGsTargetNames::SkinColor, TargetNameSkinColor);
    _targetNames.Add(EGsTargetNames::SkinShadowColor, TargetNameSkinShadowColor);
    _targetNames.Add(EGsTargetNames::SkinSpecularColor, TargetNameSkinSpecularColor);

    _targetNames.Add(EGsTargetNames::PupilColor, TargetNamePupilColor);
    _targetNames.Add(EGsTargetNames::PupilShadowColor, TargetNamePupilShadowColor);
    _targetNames.Add(EGsTargetNames::PupilSpecularColor, TargetNamePupilSpecularColor);

    _targetNames.Add(EGsTargetNames::BrowColor, TargetNameBrowColor);
    _targetNames.Add(EGsTargetNames::BrowShadowColor, TargetNameBrowShadowColor);
    _targetNames.Add(EGsTargetNames::BrowSpecularColor, TargetNameBrowSpecularColor);

    _bonesStartScaleVector.Empty();
    _scaleDisplacementPerBone.Empty();

    _facePartsId = -1;
    _hairPartsId = -1;
    _bodyPartsId = -1;
    _cloakPartsId = -1;
    _weaponPartsId = -1;
    _tattooTextureId = -1;

    SetCostume(_costumeParts.costumeId, _costumeParts.hairIsOn);
}

```

◦ 예외처리에서도 2번 호출

UGsUIWindowCustomize::OnSetPreset(int32 InIndex)

void UGsUIWindowCustomize::OnSetPreset(int32 InIndex)

```

{
    int32 prevIndex = _selectedData.GetValue(EGsUICustomizeType::PRESET_INDEX);
    if (prevIndex != InIndex)
    {
        _selectedData.SetPresetData(InIndex);
    }

    if (_customizeHandler)
    {
        _customizeHandler->Set(&_selectedData);

        // 주의: 옷이 기본장비로 리셋되므로 현재 선택 복장으로 다시 덮어써준다.
        int32 equipIndex = _toggleGroupEquip.GetSelectedIndex();
        if (_bodyList &&
            _bodyList->IsValidIndex(_toggleGroupEquip.GetSelectedIndex()))
        {
            SetBodyPartsChange(_bodyList->GetData()[equipIndex].targetId);
        }
    }

    _bIsChangePreset = true;
}

```

void UGsUIWindowCustomize::OnClickRandom()

```
{
```

```

        _selectedData.SetRandom();

    if (_customizeHandler)
    {
        _customizeHandler->Set(&_selectedData);

        // 주의: 옷이 기본장비로 리셋되므로 현재 선택 복장으로 다시 덮어써준다.
        int32 equipIndex = _toggleGroupEquip.GetSelectedIndex();
        if (_bodyList &&
            _bodyList->IsValidIndex(_toggleGroupEquip.GetSelectedIndex()))
        {
            SetBodyPartsChange(_bodyList->GetData()[equipIndex].targetId);
        }
    }
}

```

• 수정방향

- Customize, Equip, Costume 등에서는 데이터만 세팅하고 최종 외형정보는 우선순위에 의거해 1번만 처리하도록 변경

- 사용하지 않는 변수제거

mArmor ?? 이거 안쓰지 않나요?

```

struct UserLookInfo
{
public:
    ItemId          mWeapon = INVALID_ITEM_ID;
    ItemId          mArmor = INVALID_ITEM_ID;
    FairyId        mFairy = INVALID_FAIRY_ID;
    bool            mViewHelmet = true;
    CostumeId       mCostumeId = INVALID_COSTUME_ID;
    Level           mWeaponLevel = INVALID_LEVEL;
    PVPAlignmentGrade mPVPAlignmentGrade = PVPAlignmentGrade::NETURAL;

};

```

// 투구를 보여줄 것인가?

// pvp 성향 등급(이름색 표시용도)